



BIG DATA & DATA MINING

Lecturer : Kushnawi,S.Kom, M.Eng
Email : Khusnawi@amikom.ac.id
Instagram: @mynameiskhus

✓ BIG DATA & DATA MINING - WEEK 7

Objektif :

Mahasiswa dapat menerapkan metode regresi linear

Regresi linear sederhana adalah metode statistik untuk mengukur hubungan linier antara dua variabel, yaitu variabel independen dan dependen. Tujuannya adalah memprediksi nilai variabel dependen berdasarkan nilai variabel independen menggunakan garis lurus yang paling sesuai dengan data observasi. Model ini mengasumsikan bahwa terdapat hubungan sebab-akibat antara kedua variabel tersebut. Dalam regresi linear sederhana, satu variabel digunakan untuk memprediksi perubahan nilai variabel lainnya dengan menggunakan persamaan garis lurus.

Contoh kasus: Prediksi Gaji berdasarkan Tahun Bekerja

Tahapan dalam penggunaan **Simple Linear Regression** di artikel kali ini adalah sebagai berikut:

1. Import Library
2. Load Dataset (Google drive)
3. Check the General Information
4. Handling Missing Values
5. Exploratory Data Analysis (EDA)
6. Splitting Data
7. Modelling
8. Evaluation
9. Prediction

✓ 1. Import Library

```
# Import library yang dibutuhkan
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

✓ 2. Load Dataset

```
# Import dataset
url = 'https://drive.google.com/file/d/1rIR-02sw7rVdfbQA1rGtZbC2CyHowZXd/view?usp=sharing'
file_id=url.split('/')[~2]
dwn_url='https://drive.google.com/uc?id=' + file_id

dataset = pd.read_csv(dwn_url)
print(dataset.tail())
```

	Tahun_bekerja	Gaji
25	9.0	105582
26	9.5	116969
27	9.6	112635
28	10.3	122391
29	10.5	121872

✓ 3. Check the General Information

Melihat informasi Umum Ini digunakan untuk mengenali data kita.

```
#untuk melihat ukuran data
dataset.shape
```

```
(30, 2)
```

```
#Melihat informasi data kita mulai dari jumlah data, tipe data, memory yang digunakan dll.
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   Tahun_bekerja  30 non-null    float64
1   Gaji          30 non-null    int64   
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
#Melihat statistical description dari data mulai dari mean, kuartil, standard deviation dll.
dataset.describe()
```

	Tahun_bekerja	Gaji	
count	30.000000	30.000000	
mean	5.313333	76003.000000	
std	2.837888	27414.429785	
min	1.100000	37731.000000	
25%	3.200000	56720.750000	
50%	4.700000	65237.000000	
75%	7.700000	100544.750000	
max	10.500000	122391.000000	

Start coding or [generate](#) with AI.

Penjelasan Hasil `dataset.describe()`

memberikan ringkasan statistik dari data numerik dalam DataFrame. Berikut adalah penjelasan untuk setiap barisnya:

- **count:** Jumlah data non-null (tidak kosong) dalam setiap kolom.
- **mean:** Rata-rata dari nilai dalam setiap kolom.
- **std:** Standar deviasi, yang mengukur sebaran data dari rata-rata. Nilai standar deviasi yang tinggi menunjukkan data yang lebih tersebar.
- **min:** Nilai minimum dalam setiap kolom.
- **25%:** Kuartil pertama (Q1), yang merupakan nilai di bawah mana 25% data berada.
- **50%:** Median (Q2), yang merupakan nilai tengah dari data (50% data berada di bawah nilai ini).
- **75%:** Kuartil ketiga (Q3), yang merupakan nilai di bawah mana 75% data berada.
- **max:** Nilai maksimum dalam setiap kolom.

Dalam kasus dataset ini:

- **Tahun_bekerja:**
 - Terdapat 30 data `Tahun_bekerja`.
 - Rata-rata tahun bekerja adalah sekitar 5.31 tahun.
 - Standar deviasinya sekitar 2.84, menunjukkan sebaran data tahun bekerja.
 - Nilai tahun bekerja terendah adalah 1.1 tahun dan tertinggi adalah 10.5 tahun.
 - Median tahun bekerja adalah 4.7 tahun.
- **Gaji:**
 - Terdapat 30 data `Gaji`.
 - Rata-rata gaji adalah sekitar 76003.
 - Standar deviasinya sekitar 27414, menunjukkan sebaran data gaji.
 - Nilai gaji terendah adalah 37731 dan tertinggi adalah 122391.
 - Median gaji adalah 65237.

Informasi ini membantu kita memahami distribusi dan karakteristik dasar dari variabel `Tahun_bekerja` dan `Gaji` dalam dataset.

4. Handling Missing Values

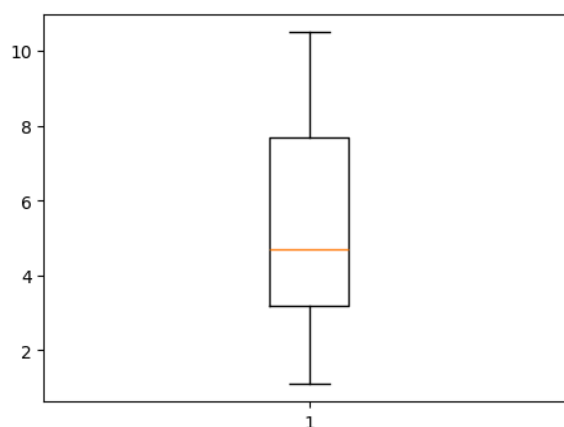
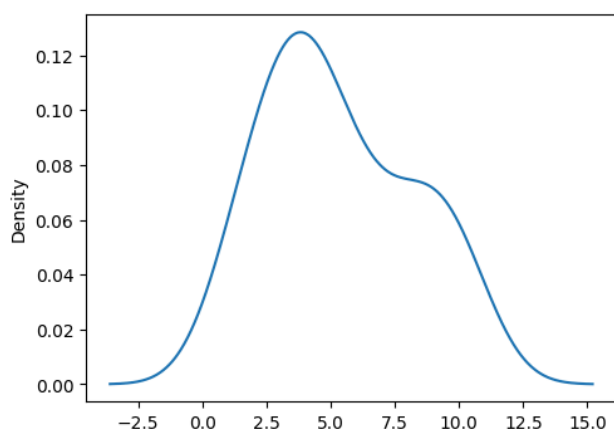
```
#Mencari dan menangani missing values.  
#Ternyata data kita tidak ada missing values.  
dataset.isnull().sum()
```

```
0  
Tahun_bekerja 0  
Gaji          0  
  
dtype: int64
```

5. Exploratory Data Analysis (EDA)

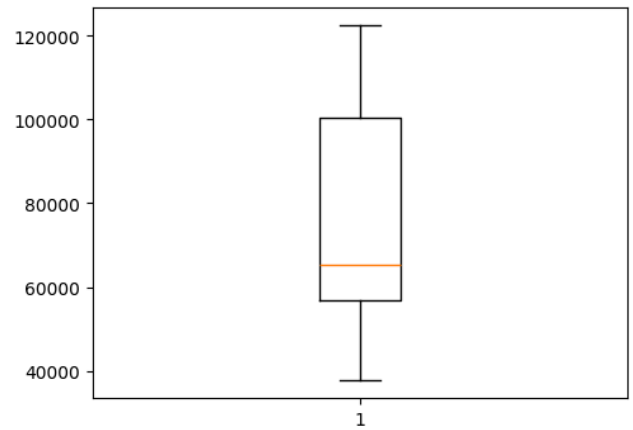
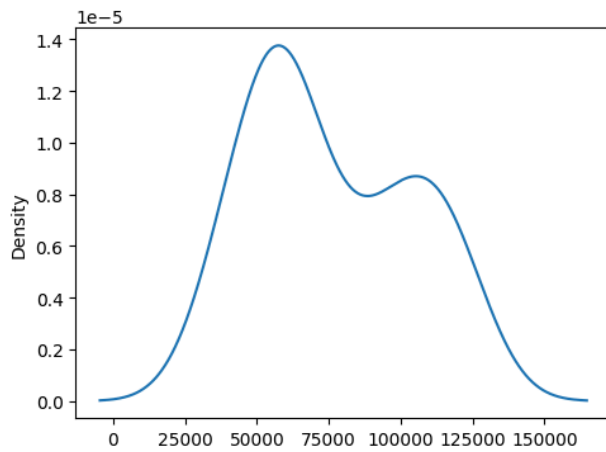
Untuk digunakan untuk lebih mengenal data kita dan menemukan insights dari data.

```
#Melakukan Exploratory Data Analysis (EDA)  
#Univariate analysis Tahun_bekerja.  
#Melihat distribusi dari Tahun_bekerja.  
f = plt.figure(figsize=(12,4))  
f.add_subplot(1,2,1)  
dataset['Tahun_bekerja'].plot(kind='kde')  
f.add_subplot(1,2,2)  
plt.boxplot(dataset['Tahun_bekerja'])  
plt.show()
```



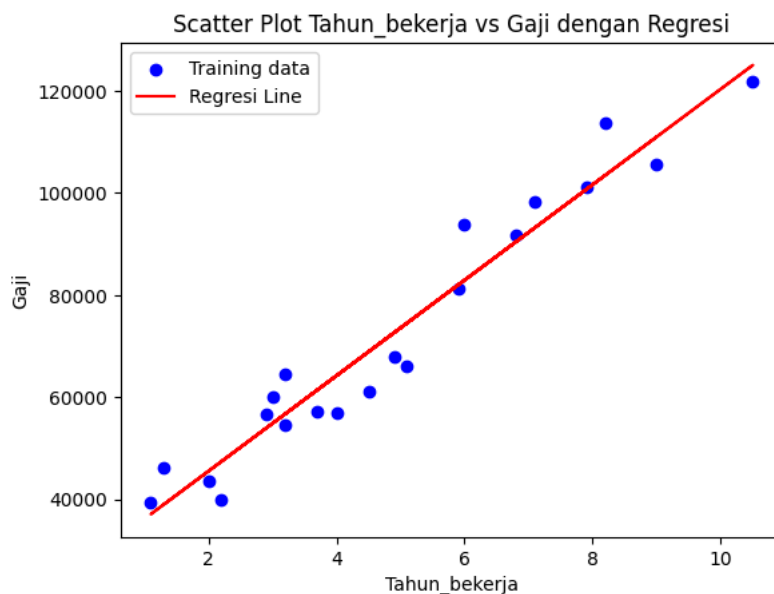
- Dapat dilihat bahwa distribusi dari `Tahun_bekerja` cenderung terkonsentrasi pada nilai-nilai yang lebih rendah, dengan puncaknya berada di sekitar 2-4 tahun. Distribusi ini sedikit miring ke kanan.
- Pada boxplot, garis tengah (median) berada di sekitar 4-5 tahun. Kotak (interquartile range/IQR) mencakup sebagian besar data antara sekitar 3 hingga 7 tahun. Tidak terdapat titik-titik di luar kumis (whiskers), menunjukkan tidak adanya outlier yang signifikan pada kolom `Tahun_bekerja`.

```
#Melakukan Exploratory Data Analysis (EDA) untuk lebih mengenal data kita dan menemukan insights dari data.  
#Univariate analysis Gaji.  
#Melihat distribusi dari Gaji.  
f = plt.figure(figsize=(12,4))  
f.add_subplot(1,2,1)  
dataset['Gaji'].plot(kind='kde')  
f.add_subplot(1,2,2)  
plt.boxplot(dataset['Gaji'])  
plt.show()
```



- Dapat dilihat bahwa mean dan median dari `Gaji` terpusat di sekitar nilai 70000-80000.
- tidak terdapat outlier/pencilan data yang bisa dilihat di boxplot.

```
#Bivariate analysis Tahun_bekerja dan Gaji.
#Menggunakan scatter plot.
plt.scatter(X_train, Y_train, color='blue', label='Training data') # Menggunakan X_train dan Y_train untuk scatt
plt.plot(X_train, lin_reg.predict(X_train), color='red', label='Regresi Line') # Menambahkan garis regresi
plt.xlabel('Tahun_bekerja')
plt.ylabel('Gaji')
plt.title('Scatter Plot Tahun_bekerja vs Gaji dengan Regresi')
plt.legend()
plt.show()
```



Hasil scatter plot menunjukkan adanya hubungan linier positif yang kuat antara `Tahun_bekerja` dan `Gaji`. Semakin lama seseorang bekerja, cenderung semakin tinggi gajinya. Titik-titik data terlihat cukup terkonsentrasi di sekitar garis lurus, mengindikasikan bahwa model regresi linier sederhana mungkin cocok untuk memprediksi gaji berdasarkan tahun bekerja.

```
#Mengetahui nilai korelasi dari Tahun_bekerja dan Gaji.
dataset.corr()
```

	Tahun_bekerja	Gaji	
Tahun_bekerja	1.000000	0.978242	il
Gaji	0.978242	1.000000	

6. Splitting Data

```
#ambil konten data saja, buang header nya
#X sebagai variabel independent/variabel bebas/variabel independent/ predictor, yakni Tahun bekerja
```

```
X = dataset.iloc[:, :-1].values
print(X)
```

```
[[ 1.1]
 [ 1.3]
 [ 1.5]
 [ 2. ]
 [ 2.2]
 [ 2.9]
 [ 3. ]
 [ 3.2]
 [ 3.2]
 [ 3.7]
 [ 3.9]
 [ 4. ]
 [ 4. ]
 [ 4.1]
 [ 4.5]
 [ 4.9]
 [ 5.1]
 [ 5.3]
 [ 5.9]
 [ 6. ]
 [ 6.8]
 [ 7.1]
 [ 7.9]
 [ 8.2]
 [ 8.7]
 [ 9. ]
 [ 9.5]
 [ 9.6]
 [10.3]
 [10.5]]
```

```
#ambil konten data saja, buang header nya
#Y sebagai variabel dependent/variable terikat/variabel tak bebas/ response, yakni Gaji
Y = dataset.iloc[:, 1].values
print(Y)
```

```
[ 39343  46205  37731  43525  39891  56642  60150  54445  64445  57189
  63218  55794  56957  57081  61111  67938  66029  83088  81363  93940
  91738  98273 101302 113812 109431 105582 116969 112635 122391 121872]
```

```
# Membagi data menjadi Training Set dan Test Set
# import skit learn untuk Machine Learning
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.3, random_state = 0)
print(X_train.shape)
print(X_train)
```

```
(21, 1)
[[ 7.9]
 [ 2.9]
 [ 5.1]
 [ 3.2]
 [ 4.5]
 [ 8.2]
 [ 6.8]
 [ 1.3]
 [10.5]
 [ 3. ]
 [ 2.2]
 [ 5.9]
 [ 6. ]
 [ 3.7]
 [ 3.2]
 [ 9. ]
 [ 2. ]
 [ 1.1]
 [ 7.1]
 [ 4.9]
 [ 4. ]]
```

7. Modelling

```
# panggil modul LinearRegression untuk menjalankan regresi linear
from sklearn.linear_model import LinearRegression
# membuat objek regresi linear
lin_reg = LinearRegression()
# membangun model (training model dari data set yang sudah kita split)
lin_reg.fit(X_train, Y_train)
```

▼ LinearRegression ⓘ ?

LinearRegression()

```
# cari tau nilai slope/koeffisien (θi ) dan intercept (θ0).
# ŷ=θ0+θiXi+εi
```

```
print(lin_reg.coef_)
print(lin_reg.intercept_)
```

```
[9360.26128619]
26777.391341197625
```

Jika rumus persamaan regresi liner sederhana : $y = a + bx$ maka dari hasil model $y = 26777,391 + 9360,261X$

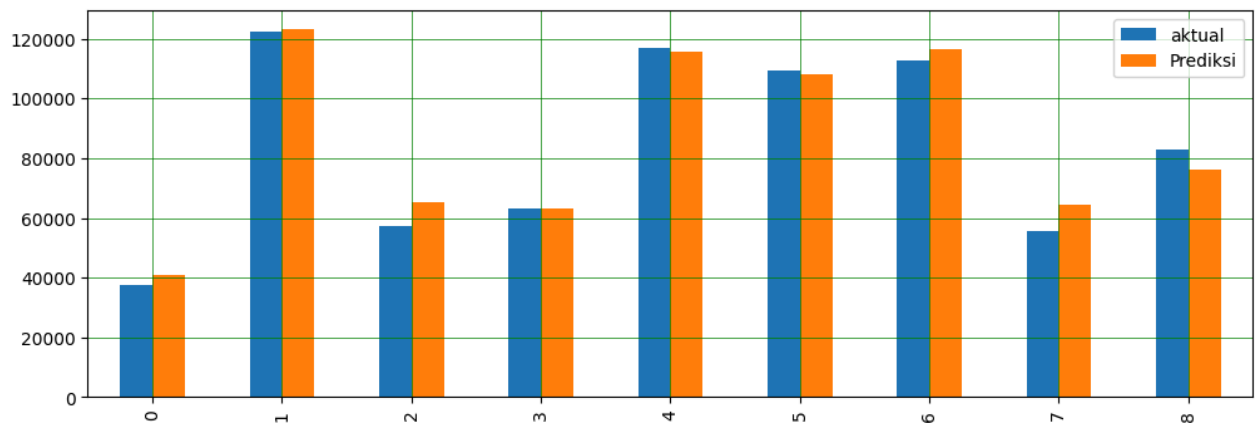
▼ 8. Evaluation

```
# memprediksi model dengan test set(melakukan proses testing)
Y_prediksi = lin_reg.predict(X_test)
```

```
#Lakukan evaluasi performa
#tampilkan hasil dari data aktual dan data prediksi
df = pd.DataFrame({'aktual': Y_test, 'Prediksi': Y_prediksi})
print(df)
```

	aktual	Prediksi
0	37731	40817.783270
1	122391	123188.082589
2	57081	65154.462615
3	63218	63282.410357
4	116969	115699.873560
5	109431	108211.664531
6	112635	116635.899689
7	55794	64218.436486
8	83088	76386.776158

```
#visualisasi galat/ selisih data aktual dengan data prediksi
df.plot(kind='bar',figsize=(12,4))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```



```
#melakukan evaluasi/ pengukuran performa algoritma
#menghitung nilai MAE, MSE dan RMSE
from sklearn import metrics
print('Mean Absolute Error:', metrics.mean_absolute_error(Y_test, Y_prediksi))
print('Mean Squared Error:', metrics.mean_squared_error(Y_test, Y_prediksi))
print('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(Y_test, Y_prediksi)))
```

```
Mean Absolute Error: 3737.417861878896
Mean Squared Error: 23370078.800832972
Root Mean Squared Error: 4834.260936361728
```

▼ Penjelasan Hasil Metrik Evaluasi

Metrik evaluasi yang ditampilkan memberikan gambaran seberapa baik model regresi linear sederhana memprediksi gaji berdasarkan tahun bekerja pada data uji (Y_{test}) dibandingkan dengan nilai prediksi oleh model ($Y_{prediksi}$).

- **Mean Absolute Error (MAE):**

- Mengukur rata-rata selisih absolut antara nilai aktual dan nilai prediksi.
- Nilai MAE sebesar 3737.417861878896 berarti, rata-rata, prediksi gaji oleh model meleset sekitar 3737.42 unit dari gaji sebenarnya. MAE memberikan gambaran langsung tentang ukuran kesalahan dalam unit yang sama dengan variabel target (Gaji).

- **Mean Squared Error (MSE):**

- Mengukur rata-rata dari kuadrat selisih antara nilai aktual dan nilai prediksi.
- Nilai MSE sebesar 23370078.800832972 memberikan bobot lebih besar pada kesalahan yang lebih besar karena dikuadratkan. MSE berguna untuk melihat seberapa besar variasi kesalahan dalam prediksi. Namun, nilainya dalam satuan kuadrat, sehingga sulit diinterpretasikan dalam konteks asli.

- **Root Mean Squared Error (RMSE):**

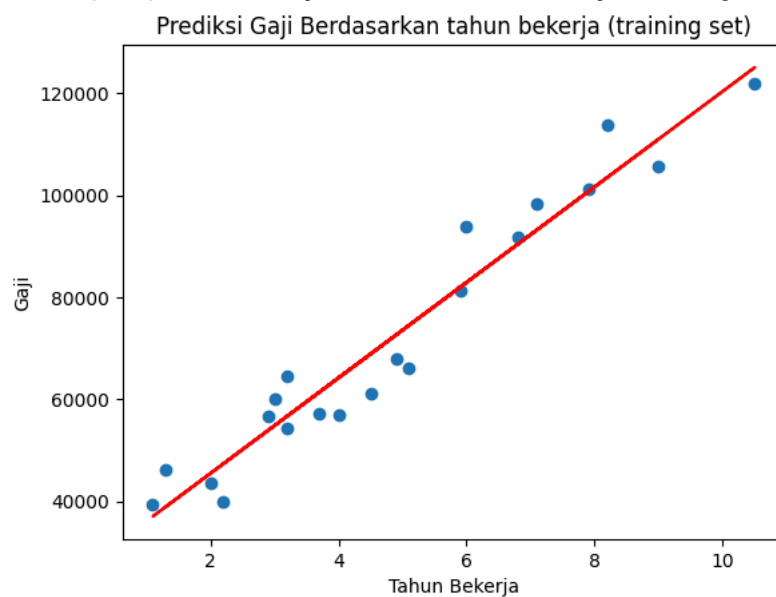
- Akar kuadrat dari MSE. Metrik ini mengembalikan unit kesalahan ke unit asli variabel target (Gaji).
- Nilai RMSE sebesar 4834.260936361728 berarti standar deviasi dari sisa (residual) adalah sekitar 4834.26. RMSE juga memberikan ukuran rata-rata kesalahan, tetapi lebih sensitif terhadap outlier dibandingkan MAE. Dalam kasus ini, rata-rata selisih antara prediksi dan nilai aktual sekitar 4834.26 unit.

Secara umum, nilai MAE, MSE, dan RMSE yang lebih rendah menunjukkan performa model yang lebih baik. Untuk mengevaluasi apakah nilai-nilai ini "baik" atau "buruk", perlu dibandingkan dengan rentang nilai gaji dalam dataset dan konteks bisnis atau masalah yang sedang diselesaikan.

```
#visualisasi data training
```

```
plt.scatter(X_train, Y_train)
plt.plot(X_train, lin_reg.predict(X_train), c='red')
plt.xlabel('Tahun Bekerja')
plt.ylabel('Gaji')
plt.title('Prediksi Gaji Berdasarkan tahun bekerja (training set)')
```

```
Text(0.5, 1.0, 'Prediksi Gaji Berdasarkan tahun bekerja (training set)')
```



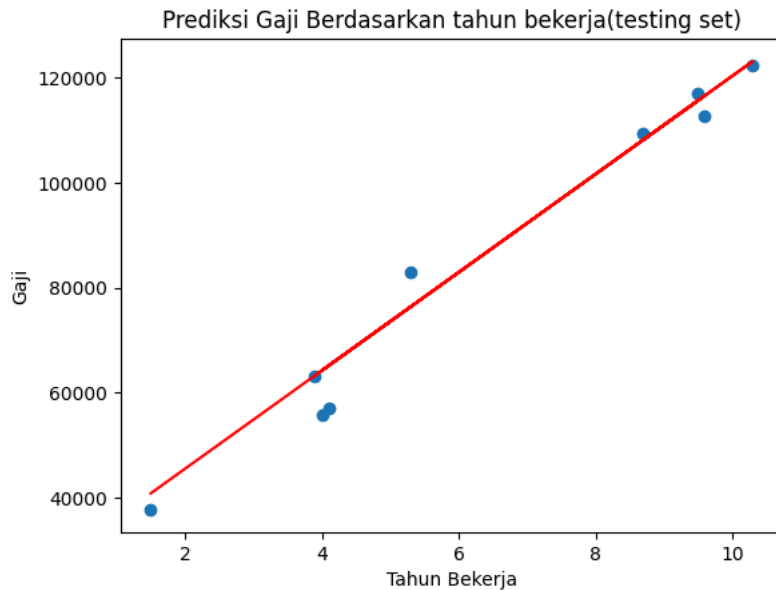
- Visualisasi ini menunjukkan titik-titik data dari **training set** (data yang digunakan untuk melatih model) dalam bentuk scatter plot, dengan **Tahun Bekerja** di sumbu X dan **Gaji** di sumbu Y. Garis berwarna merah menunjukkan garis regresi yang diperoleh dari model regresi linear yang sudah dilatih.
- Garis regresi ini adalah representasi visual dari persamaan $y = 26777,391 + 9360,261X$, di mana y adalah prediksi **Gaji** dan X adalah **Tahun Bekerja**. Garis ini menunjukkan tren linear terbaik yang ditemukan oleh model pada data training. Titik-titik data training tersebar di sekitar garis regresi, menunjukkan bahwa model ini cukup baik menangkap hubungan linear antara tahun bekerja dan gaji pada data training.

Start coding or [generate](#) with AI.

```
#visualisasi data dari data testing
plt.scatter(X_test, Y_test)
```

```
plt.plot(X_test, Y_prediksi, c='r')
plt.xlabel('Tahun Bekerja')
plt.ylabel('Gaji')
plt.title('Prediksi Gaji Berdasarkan tahun bekerja(testing set)')
```

```
Text(0.5, 1.0, 'Prediksi Gaji Berdasarkan tahun bekerja(testing set)')
```



- Visualisasi ini menampilkan titik-titik data dari **testing set** (data yang tidak digunakan untuk melatih model) dalam bentuk scatter plot, dengan **Tahun Bekerja** di sumbu X dan **Gaji** di sumbu Y. Garis berwarna merah menunjukkan garis regresi yang sama dengan yang diperoleh dari data training, tetapi sekarang ditampilkan bersama dengan data testing.
- Tujuan dari visualisasi ini adalah untuk melihat seberapa baik model yang sudah dilatih dapat memprediksi gaji pada data yang belum pernah dilihat sebelumnya (data testing). Jika titik-titik data testing juga tersebar di sekitar garis regresi, ini menunjukkan bahwa model memiliki kemampuan generalisasi yang baik dan tidak overfitting pada data training. Dalam kasus ini, titik-titik data testing terlihat cukup dekat dengan garis regresi, mengindikasikan bahwa model bekerja dengan baik pada data baru.

9. Prediction

```
# mencoba studi kasus, misal kita prediksi gaji seseorang yang telah bekerja selama 2.7, 5.5, 7.5 tahun
print('Gaji Seseorang setelah bekerja selama 2.7 tahun adalah', lin_reg.predict([[2.7]]))
print('Gaji Seseorang setelah bekerja selama 5.5 tahun adalah', lin_reg.predict([[5.5]]))
print('Gaji Seseorang setelah bekerja selama 7.5 tahun adalah', lin_reg.predict([[7.5]]))
```

```
Gaji Seseorang setelah bekerja selama 2.7 tahun adalah [52050.09681392]
Gaji Seseorang setelah bekerja selama 5.5 tahun adalah [78258.82841526]
Gaji Seseorang setelah bekerja selama 7.5 tahun adalah [96979.35098765]
```

Contoh tambahan dengan model multivariabel --> Regresi Linier Berganda

```
import pandas as pd
# Import dataset
url = 'https://drive.google.com/file/d/1zVJpdHTET8ApgsPhB3nTUDc9uwriKj3/view?usp=sharing'
file_id=url.split('/')[-2]
dwn_url='https://drive.google.com/uc?id=' + file_id
```

```
dataset = pd.read_csv(dwn_url)
print(dataset.head())
```

	Car	Model	Volume	Weight	CO2
0	Toyoty	Aygo	1000	790	99
1	Mitsubishi	Space Star	1200	1160	95
2	Skoda	Citigo	1000	929	95
3	Fiat	500	900	865	90
4	Mini	Cooper	1500	1140	105

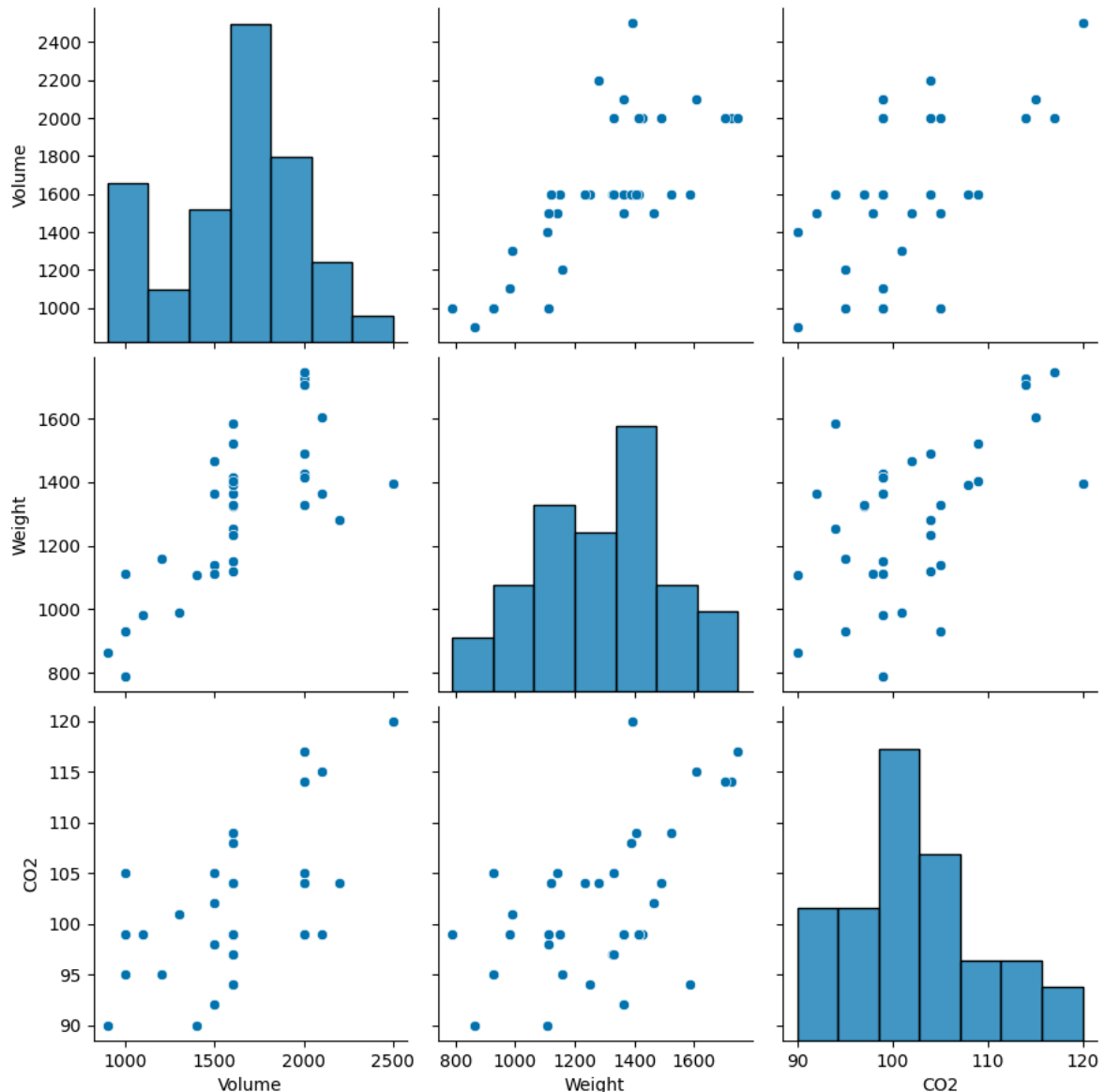
```
#mendefinisikan nilai independen dan beri nama variabel ini X dan nilai dependen ke dalam variabel bernama y
X = dataset[['Weight', 'Volume']]
y = dataset['CO2']
```

```
## Data Visualization
import seaborn as sns
```

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

```
# Visualize dengan scatter plot and histogram
sns.set_palette('colorblind')
sns.pairplot(data=dataset, height=3)
```

```
<seaborn.axisgrid.PairGrid at 0x7f8eacbcf9e0>
```



`sns.pairplot` adalah fungsi dari library Seaborn yang membuat matriks scatter plot dan histogram untuk semua pasangan variabel dalam DataFrame.

- **Scatter plot:** Plot di luar diagonal menunjukkan hubungan antara dua variabel yang berbeda. Setiap titik mewakili satu baris data, dengan posisinya ditentukan oleh nilai variabel pada sumbu X dan Y. Ini berguna untuk melihat adanya korelasi atau pola hubungan antar variabel. Dalam kasus ini, Anda bisa melihat hubungan antara `Weight` dan `Volume`, `Weight` dan `CO2`, serta `Volume` dan `CO2`.
- **Histogram:** Plot pada diagonal menunjukkan distribusi dari masing-masing variabel tunggal. Histogram menampilkan frekuensi (jumlah data) dari nilai-nilai dalam rentang tertentu. Ini berguna untuk memahami sebaran data untuk setiap variabel (`Weight`, `Volume`, dan `CO2`).

Dari grafik tersebut, Anda bisa melihat hubungan visual antara `Volume`, `Weight`, dan `CO2`. Misalnya, scatter plot antara `Weight` dan `CO2` atau `Volume` dan `CO2` mungkin menunjukkan tren atau pola tertentu yang mengindikasikan hubungan di antara variabel-variabel tersebut.

```
#Memanggil/membuat model dengan sklearn
from sklearn.linear_model import LinearRegression
regr = LinearRegression()
model=regr.fit(X, y)
print('Intercept:', model.intercept_)
```

```
print('Coefficients:', model.coef_)
```

```
Intercept: 79.69471929115939  
Coefficients: [0.00755095 0.00780526]
```

Jika dimasukkan ke rumus $y = a + b_1X_1 + b_2X_2$ maka :

$y = 79,6947 + 0,0075X_1 + 0,0078X_2$

```
#memprediksi hasil model  
#misalkan memprediksi emisi CO2 sebuah mobil yang beratnya 2300 dan volumenya 1300  
predictedCO2 = regr.predict([[2300, 1300]])  
  
print('hasil prediksi', predictedCO2)
```

```
hasil prediksi [107.2087328]  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names: Use X.columns instead  
warnings.warn(
```

PENUGASAN

1. Terapkan metode linear regression dengan menggunakan dataset StudentPerformance-2.csv minggu lalu
2. Gunakan sepasang kolom dengan korelasi tertinggi untuk dibuat nilai prediksinya --> **harus di cari dengan model, tidak selera masing-masing**
3. Berikan analisis dan kesimpulan untuk hasilnya
4. File ipynb dapat dikumpulkan pada link berikut : <https://forms.gle/zyHJ3jCCTYieAjnP8>

References

1. <https://megabagus.id>
2. <https://medium.com/@adiptamartulandi>
3. <https://stackabuse.com/linear-regression-in-python-with-scikit-learn/>
4. https://books.google.co.id/books?id=pFMOEQAAQBAJ&pg=PA117&hl=id&source=gbs_toc_r&cad=2